Análise de Programação Paralela aplicada ao Algoritmo Merge Sort

Sistemas Distribuídos / Programação Paralela – UTFPR

Matheus Xavier
matheusxavier@alunos.utfpr.edu.br

Nailson Chagas
nailsonchagas@alunos.utfpr.edu.br

8 de abril de 2026

Abstract

Este trabalho apresenta a implementação e análise de versões sequenciais e paralelas do algoritmo de ordenação Merge Sort. Foram desenvolvidas cinco versões do algoritmo: duas sequenciais (Java e C) e três paralelas (Java com *Threads*, C com *Pthreads* e C com *OpenMP*). Os experimentos foram conduzidos com arranjos de tamanho variando entre 1.000 e 150.000 elementos, e os resultados são comparados em termos de tempo de execução médio e *speedup*. Os resultados demonstram que a versão em C com OpenMP obteve o melhor desempenho geral, com *speedup* médio de 3,22, enquanto a versão Java paralela apresentou ganho mais modesto (1,64), evidenciando o impacto do *overhead* de criação de *threads* e do gerenciamento da JVM em problemas de granularidade fina — aqueles em que o trabalho por tarefa paralela é pequeno em relação ao custo de criação e sincronização de *threads*.

Palavras-chave: Programação Paralela, Merge Sort, Threads, OpenMP, Pthreads, Speedup.

1 Introdução

A crescente disponibilidade de processadores com múltiplos núcleos tornou a programação paralela uma das principais estratégias para obtenção de desempenho em aplicações computacionalmente intensivas. Algoritmos clássicos de ordenação, como o Merge Sort, são candidatos naturais à paralelização por seguirem o paradigma de divisão e conquista, no qual subproblemas podem ser resolvidos de forma independente.

Este trabalho tem por objetivo implementar, avaliar e comparar diferentes abordagens de paralelização do Merge Sort utilizando duas linguagens de programação (Java e C) e dois modelos de paralelismo em memória compartilhada (*threads* nativas/Pthreads e OpenMP). A motivação central é compreender, por meio de experimentação controlada, como a escolha da linguagem, o modelo de paralelismo e o tamanho da entrada influenciam o ganho de desempenho obtido.

2 Descrição do Problema

O Merge Sort é um algoritmo de ordenação baseado no paradigma de divisão e conquista. Dado um vetor de entrada A com n elementos, o algoritmo divide recursivamente A em duas metades, ordena cada uma delas e realiza a intercalação (*merge*) das partes ordenadas, produzindo um vetor de saída totalmente ordenado.

- **Entrada:** vetor A de n inteiros gerados aleatoriamente.
- **Saída:** vetor A' contendo os mesmos elementos de A em ordem não decrescente.
- **Complexidade:** $O(n \log n)$ no pior caso, com complexidade de espaço $O(n)$ devido ao vetor auxiliar utilizado durante a operação de *merge*.

A estrutura recursiva do algoritmo permite que as duas chamadas sobre as metades do vetor sejam executadas em paralelo, o que, em princípio, possibilita um *speedup* próximo do número de núcleos disponíveis para entradas suficientemente grandes.

3 Implementações

Foram desenvolvidas cinco versões do algoritmo, descritas a seguir. Todas compartilham o mesmo esqueleto lógico: divisão recursiva, ordenação direta por *Insertion Sort* para subvetores pequenos ($\leq$ 64 elementos) e reutilização de um buffer auxiliar previamente alocado — vetor temporário utilizado na etapa de intercalação (*merge*) para armazenar os elementos durante a fusão das metades ordenadas —, minimizando custos de alocação dinâmica.

3.1 Versão Sequencial em Java

A versão sequencial em Java implementa o Merge Sort de forma clássica. O método público `sort()` aloca o buffer auxiliar `tmp` como `new int[arr.length]` e delega à função recursiva `msort()`, que recebe o vetor original `arr`, o buffer `tmp`, e os índices `l` e `r` que delimitam a partição atual. A cada chamada, o vetor é dividido ao meio (`m = (l + r) / 2`), e as duas metades são ordenadas recursivamente de forma

sequencial — primeiro a esquerda, depois a direita. Quando o subvetor possui 64 elementos ou menos, a recursão é interrompida e aplica-se *Insertion Sort* direto, reduzindo o *overhead* de chamadas recursivas em partições pequenas. Após ambas as metades estarem ordenadas, a função `merge()` intercala os elementos utilizando o buffer `tmp` e copia o resultado de volta ao vetor original com `System.arraycopy()`. Essa versão serve como base de comparação para a versão paralela em Java.

3.2 Versão Sequencial em C

A versão sequencial em C (`msort_seq()`) segue a mesma lógica da implementação em Java: divisão recursiva, *Insertion Sort* para subvetores de até 64 elementos, e intercalação com buffer auxiliar. As diferenças residem no gerenciamento de memória — o buffer `tmp` é alocado uma única vez com `malloc()` e liberado com `free()` ao final — e na cópia do resultado, realizada com `memcpy()` em vez de `System.arraycopy()`. A versão em C beneficia-se de um modelo de execução mais previsível, da ausência da sobrecarga da JVM e das otimizações do compilador GCC (flag `-O3`), resultando em desempenho significativamente superior ao da versão equivalente em Java.

3.3 Versão Paralela com *Threads* em Java

A paralelização em Java foi implementada utilizando a classe `Thread` da API padrão. A estratégia segue o modelo *fork-join* adaptativo: enquanto a profundidade restante for maior que zero (`depth > 0`), duas *threads* independentes são criadas via `new Thread(() -> msort(...))` para processar a metade esquerda e a metade direita do vetor. Ambas são iniciadas com `t1.start()` e `t2.start()` antes de qualquer chamada a `t1.join()` e `t2.join()`, maximizando o paralelismo entre as partições. A *thread* chamadora bloqueia até que ambas as filhas terminem (com tratamento de `InterruptedException` via `try-catch`), executando então a etapa de *merge* sobre as duas metades ordenadas. Quando `depth == 0`, as chamadas recursivas são realizadas sequencialmente, sem criação de novas *threads*.

A profundidade máxima de paralelização (`maxDepth`) é determinada pelo menor inteiro tal que $2^{\texttt{maxDepth}} \geq$ cores, onde o número de processadores lógicos é obtido via `Runtime.getRuntime().availableProcessors()`. Na prática, isso limita o número máximo de *threads* folha executando simultaneamente a $2^{\texttt{maxDepth}}$. Para o Ryzen 5 2600 (12 *threads* lógicas), `maxDepth` $= 4$ e até $2^4 = 16$ *threads* folha podem estar executando trabalho útil simultaneamente nos níveis mais profundos da recursão. Para subvetores com até 64 elementos, utiliza-se *Insertion Sort* direto, evitando o *overhead* de chamadas recursivas adicionais.

3.4 Versão Paralela com *Pthreads* em C

A implementação em C com Pthreads segue a mesma estratégia *fork-join* adaptativa da versão Java. A cada nível de recursão em que `depth > 0`, duas *threads* POSIX são criadas explicitamente via `pthread_create()`, cada uma recebendo uma estrutura `thread_args_t` contendo: o ponteiro para o vetor (`arr`), o buffer auxiliar compartilhado (`tmp`), os índices esquerdo e direito da partição (`l`, `r`) e a profundidade restante (`depth`). Uma função wrapper (`thread_func`) desempacota esses argumentos e invoca recursivamente a função `msort_pt()` com `depth - 1`. Após a criação de ambas as *threads*, a *thread* pai bloqueia em `pthread_join(t1, NULL)` e `pthread_join(t2, NULL)`, garantindo que ambas as metades estejam ordenadas antes de executar o *merge*. Quando `depth == 0`, ambas as metades são processadas sequencialmente pela mesma *thread*, sem criação de novas.

O limite de profundidade (`max_depth`) é o menor inteiro tal que $2^{\texttt{max_depth}} \geq n_{\text{cores}}$, onde n_{cores} é obtido via `sysconf(_SC_NPROCESSORS_ONLN)`. O buffer auxiliar `tmp` é alocado uma única vez com `malloc(n * sizeof(int))`, liberado com `free()` ao final, e compartilhado entre todas as *threads* — o que é seguro pois cada *thread* opera em faixas de índices disjuntas do vetor. Ao fim de cada chamada recursiva, a função `merge()` intercala as duas metades ordenadas utilizando `tmp` como espaço temporário e copia o resultado de volta ao vetor original com `memcpy()`.

3.5 Versão Paralela com OpenMP em C

A versão com OpenMP adota o modelo de tarefas (`#pragma omp task`), estruturado em duas camadas. Na camada externa, a região paralela é iniciada com `#pragma omp parallel`, que cria um *thread pool* cujo tamanho é determinado por `omp_get_max_threads()`. Dentro dessa região, a diretiva `#pragma omp single nowait` garante que apenas uma *thread* execute a chamada inicial a `msort()`, enquanto as demais ficam ociosas no pool, prontas para receber tarefas. Na camada interna (função recursiva `msort()`), enquanto `depth > 0`, cada chamada sobre uma metade do vetor é encapsulada em uma diretiva `#pragma omp task shared(arr, tmp)`, que cria uma tarefa assíncrona com acesso compartilhado ao vetor e ao buffer auxiliar. Em seguida, `#pragma omp taskwait` atua como barreira, suspendendo a execução até que ambas as tarefas filhas (metade esquerda e metade direita) tenham sido concluídas, permitindo então a execução segura do *merge*. Quando `depth == 0`, a recursão prossegue sequencialmente, sem criação de novas tarefas.

O limite de profundidade (`max_depth`) segue o mesmo critério das versões anteriores: o menor inteiro

tal que $2^{\texttt{max-depth}} \geq$ `omp_get_max_threads()`. A principal diferença em relação à versão com Pthreads é que nenhuma *thread* nova é criada ou destruída durante a execução — todas as *threads* são alocadas na abertura da região paralela e reutilizadas ao longo de toda a recursão. As tarefas criadas com `#pragma omp task` são enfileiradas e distribuídas entre as *threads* do pool pelo *runtime* do OpenMP, o que reduz significativamente o *overhead* em comparação à criação explícita de *threads* via `pthread_create()`.

4 Metodologia de Avaliação

4.1 Ambiente de Testes

Os experimentos foram realizados em um computador pessoal com as seguintes especificações de hardware e software:

- **Processador:** AMD Ryzen 5 2600 (6 núcleos físicos, 12 *threads* lógicas, até 3,9 GHz)
- **Memória RAM:** 64 GiB
- **Placa-mãe:** Gigabyte B450M DS3H V2
- **Sistema Operacional:** Pop!_OS 22.04 LTS (Linux, 64-bit, X11)

Foram utilizados o compilador GCC (com `-O3` e suporte a OpenMP e Pthreads) para as versões em C e o OpenJDK para as versões em Java. Cada configuração foi executada 10 vezes para cada tamanho de entrada, e reportam-se a média e o desvio padrão do tempo de execução.

4.2 Métricas

O desempenho foi avaliado com base no tempo médio de execução (em milissegundos) e no *speedup*, calculado em relação à versão sequencial da mesma linguagem:

$$Speedup = \frac{T_s}{T_p} \tag{1}$$

onde T_s é o tempo médio de execução da versão sequencial e T_p é o tempo médio de execução da versão paralela correspondente. Foram testados 150 tamanhos de entrada, variando de 1.000 até 150.000 elementos, com passo de 1.000.

5 Resultados

5.1 Tempo de Execução

A Tabela 1 apresenta o tempo médio de execução das cinco versões implementadas para alguns tamanhos representativos de entrada. Os valores estão em milissegundos.

Table 1: Tempo médio de execução (ms).

N	J-Seq	J-Par	C-Seq	C-PT	C-OMP
1.000	0,48	4,36	0,05	1,54	0,52
10.000	0,96	16,22	0,56	1,22	0,67
50.000	4,33	2,96	3,25	1,79	1,15
100.000	8,84	4,01	6,89	2,56	1,88
150.000	14,10	5,35	10,79	3,25	2,76

5.2 Speedup

A Tabela 2 mostra o *speedup* obtido por cada versão paralela em relação à respectiva versão sequencial.

Table 2: Speedup por tamanho de entrada.

N	Java Par	C Pthread	C OpenMP
1.000	0,11	0,03	0,24
10.000	0,09	0,46	1,72
50.000	1,47	1,84	2,93
100.000	2,22	2,71	3,69
150.000	2,64	3,32	3,94

Considerando todo o intervalo testado (1.000 a 150.000 elementos), os *speedups* médios obtidos foram: **C OpenMP** = 3,22; **C Pthread** = 2,08; **Java Paralelo** = 1,64.

5.3 Gráficos

A Figura 1 apresenta a comparação do tempo de execução das cinco versões em função do tamanho da entrada, enquanto a Figura 2 mostra a evolução do *speedup*.

6 Discussão

Os resultados evidenciam diferentes comportamentos de acordo com o tamanho da entrada e o modelo de paralelismo empregado. Para entradas pequenas (até cerca de 8.000 elementos), a versão **sequencial em C** foi a mais eficiente, superando inclusive as versões paralelas. Isso ocorre porque, nesses tamanhos, o tempo de execução é tão reduzido que o custo de criação e sincronização de *threads* supera o ganho obtido pelo processamento paralelo.

A partir de aproximadamente **9.000 elementos**, a versão em C com OpenMP passa a superar consistentemente a sequencial, estabelecendo-se como a implementação dominante para entradas médias e grandes. No agregado, das 150 faixas testadas, o OpenMP venceu em 140 casos, enquanto a versão sequencial em C venceu nos 10 menores tamanhos.

A versão com **Pthreads** apresentou *speedup* inferior ao OpenMP, o que pode ser atribuído ao fato de que a criação e destruição explícita de *threads* POSIX em cada chamada recursiva (`pthread_create` + `pthread_join`) impõe um custo significativamente maior do que o modelo de tarefas do OpenMP, que

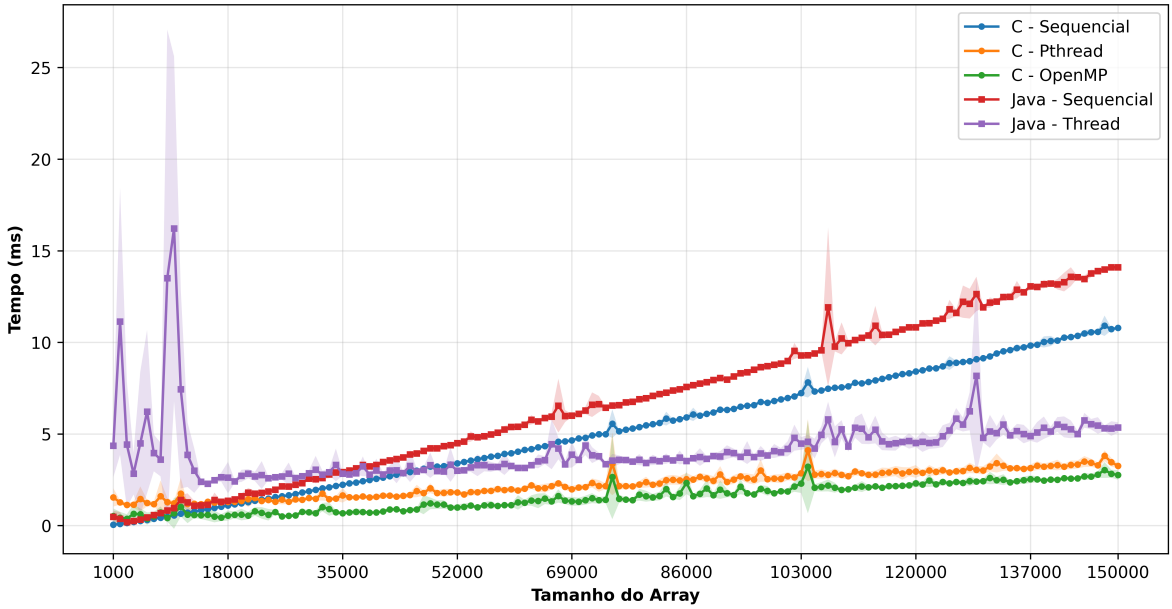

Figure 1: Tempo de execução em função do tamanho do vetor.

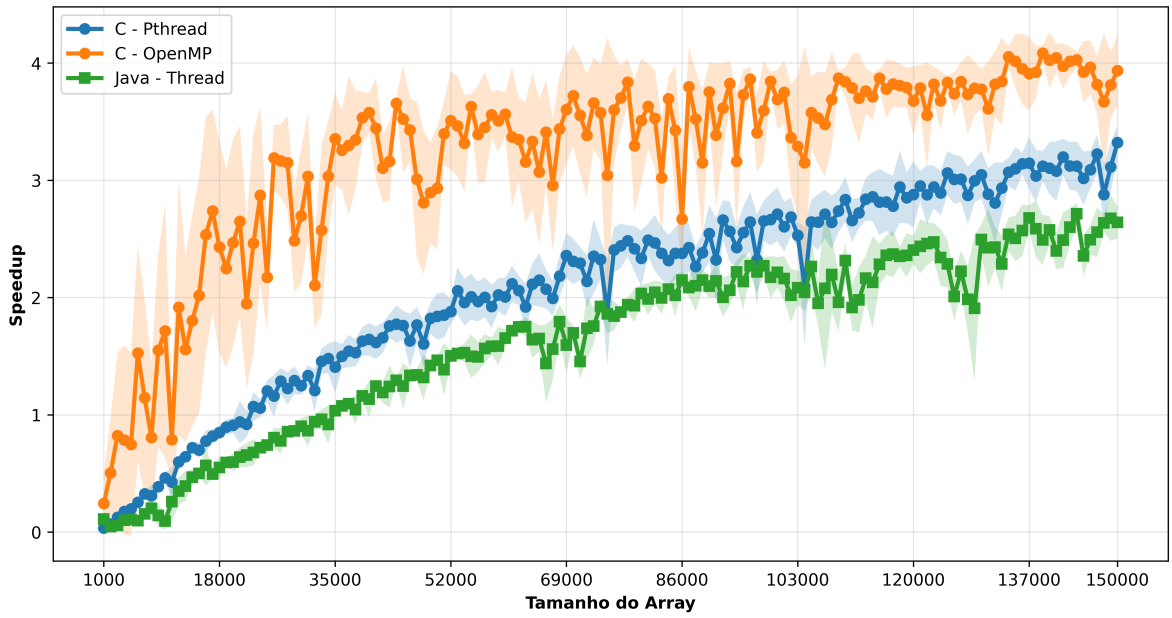

Figure 2: Speedup em função do tamanho do vetor.

reutiliza um *thread pool* fixo criado uma única vez na abertura da região paralela.

Em relação ao **Java**, observa-se um ganho real de desempenho com a paralelização (*speedup* médio 1,64), mas significativamente abaixo das versões em C. Isso se deve ao custo de criação de *threads* na JVM, à sobrecarga do *garbage collector* e à ausência de otimizações agressivas como as aplicadas pelo GCC. Curiosamente, para entradas muito pequenas, a versão paralela em Java chega a ser mais lenta que a sua respectiva versão sequencial, com *speedup* inferior a 1 — comportamento observado exclusivamente na comparação Java paralelo vs. Java sequencial. Não foi possível determinar com exatidão o motivo desse fenômeno, porém acredita-se que esteja relacionado à forma como a JVM gerencia a criação e o escalonamento de *threads*, cujo custo supera amplamente o ganho computacional em tarefas de granularidade muito fina.

No aspecto da **escalabilidade**, todas as versões paralelas apresentam crescimento do *speedup* conforme aumenta o tamanho da entrada, confirmando que o paralelismo é mais eficaz quando há trabalho suficiente para compensar seu *overhead*. O OpenMP, em particular, aproxima-se de um *speedup* de 4 em entradas maiores, indicando bom aproveitamento dos núcleos disponíveis.

7 Dificuldades Encontradas

Durante o desenvolvimento do trabalho, algumas dificuldades se destacaram:

- **Controle da profundidade de paralelismo:** determinar o limite ideal para a recursão paralela exigiu experimentação. Profundidades excessivas geravam *threads* demais e degradavam o desempenho.
- **Sincronização em Pthreads:** o uso explícito de `pthread_create` e `pthread_join` tornou o código mais verboso e suscetível a erros de concorrência, especialmente no compartilhamento do buffer auxiliar.
- **Medições em entradas pequenas:** os tempos de execução medidos em arranjos pequenos apresentaram alta variância (desvios padrão relativamente grandes), dificultando a comparação justa entre as versões nesses cenários.

8 Conclusão

Este trabalho apresentou a implementação e a análise comparativa de cinco versões do algoritmo Merge Sort, explorando abordagens sequenciais e paralelas em Java e C. Os resultados experimentais permitem destacar as seguintes conclusões:

- Para entradas pequenas (abaixo de ∼9.000 elementos), a versão sequencial em C foi a mais eficiente, demonstrando que o *overhead* do paralelismo pode superar seus benefícios em problemas de granularidade fina.
- A versão em C com **OpenMP** obteve o melhor desempenho geral, com *speedup* médio de 3,22 e dominância em 140 das 150 faixas de tamanho testadas.
- A versão com Pthreads obteve *speedup* médio de 2,08, ficando atrás do OpenMP principalmente pelo custo de criação explícita de *threads*.
- A paralelização em Java produziu ganho real (*speedup* médio de 1,64), porém inferior às versões em C devido ao custo adicional da JVM.

Como trabalhos futuros, sugere-se estender a análise para entradas ainda maiores, investigar o comportamento das implementações em sistemas com maior número de núcleos, avaliar o uso de `ForkJoinPool` em Java (que tende a reduzir o *overhead* de criação de *threads*, pois reutiliza um pool fixo de trabalhadores em vez de criar e destruir *threads* a cada chamada recursiva) e explorar modelos de paralelismo baseados em GPU para o problema de ordenação.

Repositório

O código-fonte completo das implementações está disponível em: `https://github.com/NailsonChagas/Sistemas-Distribuidos/tree/main/Trabalho%201`

References

[1] TANENBAUM, A. S.; VAN STEEN, M. *Distributed Systems: Principles and Paradigms.* 2. ed. Pearson, 2007.

[2] CORMEN, T. H. et al. *Introduction to Algorithms.* 3. ed. MIT Press, 2009.

[3] OpenMP Architecture Review Board. *OpenMP Application Programming Interface Specification*, v. 5.2, 2021.

[4] BUTENHOF, D. R. *Programming with POSIX Threads.* Addison-Wesley, 1997.